



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMATIKOS STUDIJŲ PROGRAMA

Robotikos laboratorinis darbas

Laboratorinis darbas nr. 1

Karolis Ribačonka
3 kursas 2 grupė 2 pogrupis

Vilnius
2026

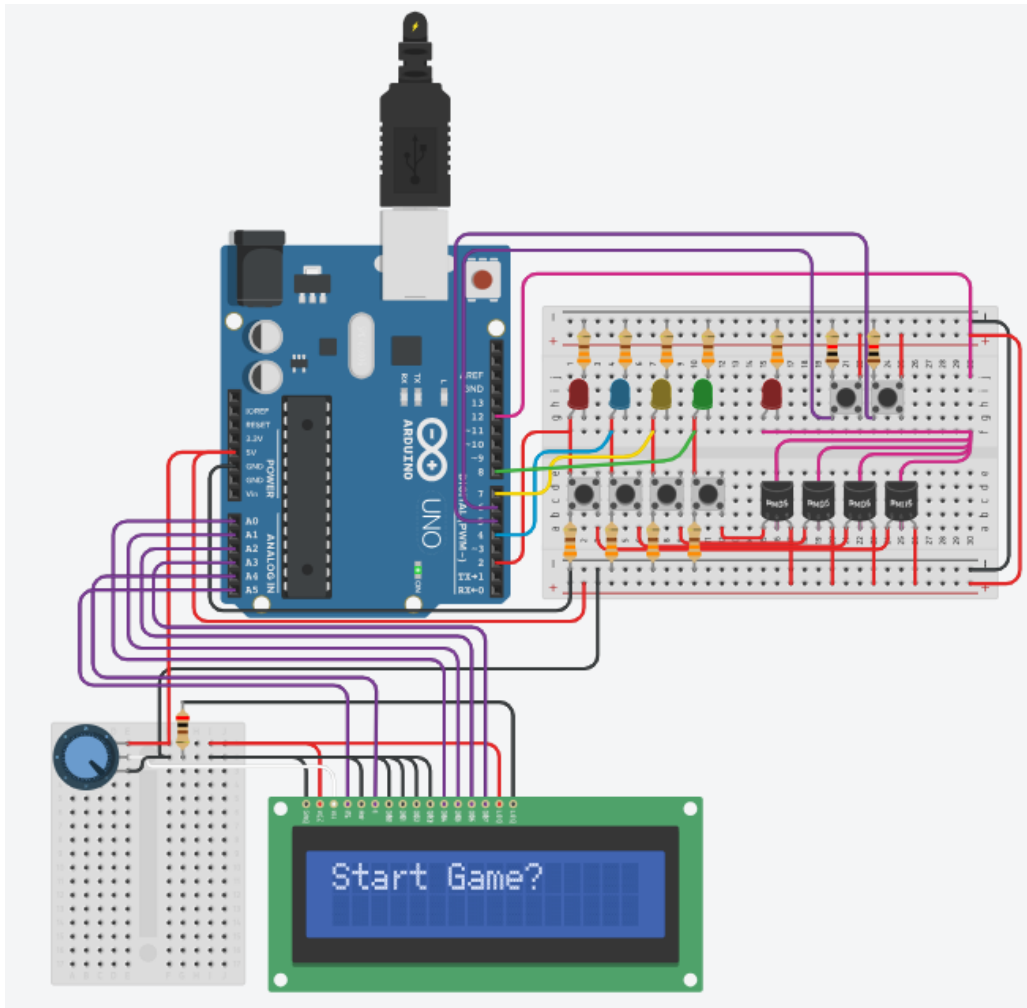
Turinys

Aprašymas	3
Detalių sąrašas	4
Išvados	5
Priedas	6

Aprašymas

Šiam laboratoriniui darbui sukūriau paprastą atminties žaidimo "Simon" kopiją su LCD ekranu, skirtu žaidimo konfigūravimui ir vartotojo sąsajai.

Žaidimas prasideda, kai yra pasirenkamas žaidimo sunkumo lygis. Kiekvieno turo metu užsidega švieselė nurodanti, kad šiuo metu vyksta rodymo fazė ir žaidėjo mygtukų paspaudimai nėra registruojami (tai yra padaroma tranzistorių pagalba). Žaidėjas turi teisinga eilės tvarka paspausti mygtukus po atitinkamomis švieselėmis ir taip daryti iki žaidimo pabaigos. Neteisingai paspaudus mygtuką žaidimas yra užbaigiamas ir ekrane matoma pralaimėjimo žinutė.



1 pav. Apraratūros schema Tinkercad sistemoje.

Detalių sąrašas

Laboratoriniui darbui atlikti buvo panaudotos šios detalės:

- Arduino Uno R3
- 5x LED lemputės
- 6x Mygtukai
- 9x 330Ω rezistoriai
- 2x $1k\Omega$ rezistoriai
- 1x 200Ω rezistorius
- 4x PMOS tranzistoriai
- Jungiamieji laidai
- Potenciometras (nustatyti ekrano kontrastui)
- 16x2 LCD ekranas

Išvados

Laboratorinio darbo metu pavyko pilnai įgyvendinti žaidimo logiką, sudėtingumo pasirinkimą ir atitinkamai parodyti laimėjimo / pralaimėjimo būsenas.

Neliko neįgyventintų dalių. Vienintelis patobulinimas būtų aiškesnė kodo struktūra.

Priedas

```
1
2 // C++ code
3 //
4
5 int PIN_USER_OK = 6;
6 int PIN_USER_OTHER = 5;
7
8 int CONTROL_PIN = 12;
9
10 int LIGHT_PINS[4] = {2, 4, 7, 8};
11
12 int lights[15];
13
14 void gamePrep(
15     int rounds
16 ) {
17
18     // Random generate which leds will occur
19     for(int i = 0; i < rounds; i++) {
20         switch(random(0, 4)){
21             case 0:
22                 lights[i] = LIGHT_PINS[0];
23                 break;
24             case 1:
25                 lights[i] = LIGHT_PINS[1];
26                 break;
27             case 2:
28                 lights[i] = LIGHT_PINS[2];
29                 break;
30             case 3:
31                 lights[i] = LIGHT_PINS[3];
32                 break;
33         }
34     }
35 }
36
37 void roundSetup(
38     int round
39 ) {
40     digitalWrite(CONTROL_PIN, HIGH);
41 }
```

```

42   delay(1000);
43
44   for(int i = 0; i < 4; i++) {
45       pinMode(LIGHT_PINS[i], OUTPUT);
46   }
47
48   for(int i = 0; i <= round; i++) {
49       digitalWrite(lights[i], HIGH);
50       delay(500);
51       digitalWrite(lights[i], LOW);
52       delay(500);
53   }
54
55   for(int i = 0; i < 4; i++) {
56       pinMode(LIGHT_PINS[i], INPUT);
57   }
58
59   digitalWrite(CONTROL_PIN, LOW);
60 }
61
62 bool roundLoop(
63     int round
64 ) {
65     int guessIndex = 0;
66
67     int currentPin = lights[guessIndex];
68
69     int prevStates[4] = {LOW, LOW, LOW, LOW};
70
71     int redInputPrevious = LOW,
72         blueInputPrevious = LOW,
73         yellowInputPrevious = LOW,
74         greenInputPrevious = LOW;
75
76     while(guessIndex <= round) {
77
78         // Check every pin
79         for(int i = 0; i < 4; i++){
80             int state = digitalRead(LIGHT_PINS[i]);
81
82             // Button has been pressed (no debounce)
83             if(state == HIGH && prevStates[i] == LOW){
84                 prevStates[i] = HIGH;

```

```

85
86     // Wrong button was pressed
87     if(currentPin != LIGHT_PINS[i]){
88         return false;
89     }
90 }
91
92     // The button has been released (this will be the correct button)
93     if(state == LOW && prevStates[i] == HIGH){
94         prevStates[i] = LOW;
95         guessIndex++;
96         currentPin = lights[guessIndex];
97     }
98 }
99 }
100
101     return true;
102 }
103
104 bool gameLoop(
105     int rounds
106 ) {
107     gamePrep(rounds);
108     for(int round = 0; round < rounds; round++){
109         roundSetup(round);
110         int roundWin = roundLoop(round);
111         if(!roundWin) {
112             return false;
113         }
114     }
115     return true;
116 }
117
118 // LCD
119
120 int lcdPins[4] = {A3, A2, A1, A0};
121 int lcdEN = A4;
122 int lcdRS = A5;
123
124 char command[5];
125 /*
126 0 - DB7
127 1 - DB6

```

```

128 2 - DB5
129 3 - DB4
130 4 - RS
131 */
132
133 void lcd_SendCommand(){
134     digitalWrite(lcdRS, command[4]);
135     for(int i = 0; i < 4; i++) {
136         digitalWrite(lcdPins[i], command[i]);
137     }
138
139     delayMicroseconds(100);
140     digitalWrite(lcdEN, HIGH);
141     delayMicroseconds(100);
142     digitalWrite(lcdEN, LOW);
143 }
144
145 void lcd_CommandEnable(){
146
147     // Set RS
148     command[4] = 0;
149
150     command[0] = 0;
151     command[1] = 0;
152     command[2] = 1;
153     command[3] = 0;
154     lcd_SendCommand();
155
156     // Send command again to set multiline
157
158     command[0] = 0;
159     command[1] = 0;
160     command[2] = 1;
161     command[3] = 0;
162     lcd_SendCommand();
163
164     command[0] = 1;
165     command[1] = 0;
166     command[2] = 0;
167     command[3] = 0;
168     lcd_SendCommand();
169 }
170

```

```

171 void lcd_CommandClear(){
172     command[4] = 0;
173
174     command[0] = 0;
175     command[1] = 0;
176     command[2] = 0;
177     command[3] = 0;
178     lcd_SendCommand();
179
180     command[0] = 0;
181     command[1] = 0;
182     command[2] = 0;
183     command[3] = 1;
184     lcd_SendCommand();
185
186     delayMicroseconds(2000);
187 }
188
189 // Using DDRAM AD SET
190 void lcd_CmdLine1(){
191
192     command[4] = 0;
193
194     command[0] = 1;
195     command[1] = 0;
196     command[2] = 0;
197     command[3] = 0;
198     lcd_SendCommand();
199
200     command[0] = 0;
201     command[1] = 0;
202     command[2] = 0;
203     command[3] = 0;
204     lcd_SendCommand();
205 }
206
207 // Using DDRAM AD SET
208 void lcd_CmdLine2(){
209     command[4] = 0;
210
211     command[0] = 1;
212     command[1] = 1;
213     command[2] = 0;

```

```

214     command[3] = 0;
215     lcd_SendCommand();
216
217     command[0] = 0;
218     command[1] = 0;
219     command[2] = 0;
220     command[3] = 0;
221     lcd_SendCommand();
222 }
223
224 void lcd_CommandDisplayOn(){
225
226     command[4] = 0;
227
228     command[0] = 0;
229     command[1] = 0;
230     command[2] = 0;
231     command[3] = 0;
232     lcd_SendCommand();
233
234     command[0] = 1;
235     command[1] = 1;
236     command[2] = 0;
237     command[3] = 0;
238     lcd_SendCommand();
239
240 }
241
242 enum lcdShiftSC {
243     LCD_SHIFT_CURSOR = 0,
244     LCD_SHIFT_SCREEN = 1
245 };
246 enum lcdShiftLR {
247     LCD_SHIFT_LEFT = 0,
248     LCD_SHIFT_RIGHT = 1
249 };
250
251 void lcd_CommandShift(
252     char screen,
253     char right
254 ) {
255     command[4] = 0;
256

```

```

257     command[0] = 0;
258     command[1] = 0;
259     command[2] = 0;
260     command[3] = 1;
261     lcd_SendCommand();
262
263     command[0] = screen;
264     command[1] = right;
265     command[2] = 0;
266     command[3] = 0;
267     lcd_SendCommand();
268 }
269
270 void lcd_DisplayStart(){
271     lcd_CommandClear();
272     lcd_CmdLine1();
273
274     String str = "Start Game?";
275
276     for(int i = 0; i < str.length(); i++) {
277         lcd_CommandPrint(str[i]);
278     }
279
280 }
281
282 typedef enum Difficulty {
283     DIFF_EASY = 0,
284     DIFF_MEDIUM = 1,
285     DIFF_HARD = 2
286 };
287 void lcd_DisplayDifficulty(
288     char difficulty
289 ){
290
291     lcd_CommandClear();
292     lcd_CmdLine1();
293
294     String diff = "Difficulty:";
295     String easy = "Easy";
296     String medium = "Medium";
297     String hard = "Hard";
298
299     for(int i = 0; i < diff.length(); i++) {

```

```

300         lcd_CommandPrint(diff[i]);
301     }
302
303     lcd_CmdLine2();
304
305     switch(difficulty) {
306         case DIFF_EASY:
307             for(int i = 0; i < easy.length(); i++) {
308                 lcd_CommandPrint(easy[i]);
309             }
310             break;
311
312         case DIFF_MEDIUM:
313             for(int i = 0; i < medium.length(); i++) {
314                 lcd_CommandPrint(medium[i]);
315             }
316             break;
317
318         case DIFF_HARD:
319             for(int i = 0; i < hard.length(); i++) {
320                 lcd_CommandPrint(hard[i]);
321             }
322             break;
323     }
324 }
325
326 void lcd_DisplayWin(){
327     lcd_CommandClear();
328     lcd_CmdLine1();
329
330     String str = "You Win!";
331
332     for(int i = 0; i < str.length(); i++) {
333         lcd_CommandPrint(str[i]);
334     }
335
336     lcd_CmdLine2();
337     String str2 = "Play again?";
338     for(int i = 0; i < str2.length(); i++) {
339         lcd_CommandPrint(str2[i]);
340     }
341
342 }

```

```

343
344 void lcd_DisplayLose(){
345     lcd_CommandClear();
346     lcd_CmdLine1();
347
348     String str = "You Lose!";
349
350     for(int i = 0; i < str.length(); i++) {
351         lcd_CommandPrint(str[i]);
352     }
353
354     lcd_CmdLine2();
355     String str2 = "Play again?";
356     for(int i = 0; i < str2.length(); i++) {
357         lcd_CommandPrint(str2[i]);
358     }
359 }
360
361
362 void lcd_DisplayPlaying(){
363
364     lcd_CommandClear();
365     lcd_CmdLine1();
366
367     String str = "Playing...";
368
369     for(int i = 0; i < str.length(); i++) {
370         lcd_CommandPrint(str[i]);
371     }
372
373 }
374
375 void lcd_CommandPrint(char chr){
376
377     command[4] = 1;
378
379     command[0] = (chr >> 7) & 0x01;
380     command[1] = (chr >> 6) & 0x01;
381     command[2] = (chr >> 5) & 0x01;
382     command[3] = (chr >> 4) & 0x01;
383     lcd_SendCommand();
384
385     command[0] = (chr >> 3) & 0x01;

```

```

386     command[1] = (chr >> 2) & 0x01;
387     command[2] = (chr >> 1) & 0x01;
388     command[3] = (chr & 0x01);
389     lcd_SendCommand();
390 }
391
392 void lcd_Setup() {
393
394     // Set pin modes
395     pinMode(lcdEN, OUTPUT);
396     pinMode(lcdRS, OUTPUT);
397     for(int i = 0; i < 4; i++) {
398         pinMode(lcdPins[i], OUTPUT);
399     }
400
401     // Enable 4-bit mode
402     lcd_CommandEnable();
403     lcd_CommandClear();
404     lcd_CmdLine1();
405     lcd_CommandDisplayOn();
406 }
407
408 // MAIN
409
410 void setup() {
411     pinMode(CONTROL_PIN, OUTPUT);
412     pinMode(PIN_USER_OK, INPUT);
413     pinMode(PIN_USER_OTHER, INPUT);
414     Serial.begin(9600);
415     lcd_Setup();
416 }
417
418 /*
419 0 - start question
420 1 - difficulty select
421 2 - game
422 3 - game win
423 4 - game lost
424 */
425 char gameState = 0;
426 bool needDisplay = true;
427
428 int pinUserOkStatePrev = LOW;

```

```

429 int pinUserOtherStatePrev = LOW;
430 int difficulty = 0;
431
432 bool gameWin = false;
433 int rounds = 0;
434
435 void loop() {
436
437     int pinUserOkState = digitalRead(PIN_USER_OK);
438     int pinUserOtherState = digitalRead(PIN_USER_OTHER);
439
440     switch(gameState){
441
442     case 0:
443         if(needDisplay){
444             lcd_DisplayStart();
445             needDisplay = false;
446         }
447
448         if(pinUserOkState == HIGH && pinUserOkStatePrev == LOW) {
449             needDisplay = true;
450             gameState = 1;
451         }
452
453     break;
454
455     case 1:
456
457         if(needDisplay) {
458             lcd_DisplayDifficulty(difficulty);
459             needDisplay = false;
460         }
461
462         if(pinUserOkState == HIGH && pinUserOkStatePrev == LOW) {
463             needDisplay = true;
464             gameState = 2;
465         }
466
467         if(pinUserOtherState == HIGH && pinUserOtherStatePrev == LOW) {
468             needDisplay = true;
469             difficulty = (difficulty + 1) % 3;
470         }
471

```

```

472     break;
473
474     case 2:
475
476         lcd_DisplayPlaying();
477         rounds = (difficulty + 1) * 2;
478         gameWin = gameLoop(rounds);
479         if(gameWin) {
480             gameState = 3;
481         }
482         else {
483             gameState = 4;
484         }
485         needDisplay = true;
486
487     break;
488
489     case 3:
490
491         if(needDisplay){
492             lcd_DisplayWin();
493             needDisplay = false;
494         }
495
496         if(pinUserOkState == HIGH && pinUserOkStatePrev == LOW) {
497             needDisplay = true;
498             gameState = 0;
499         }
500
501     break;
502
503     case 4:
504
505         if(needDisplay){
506             lcd_DisplayLose();
507             needDisplay = false;
508         }
509
510         if(pinUserOkState == HIGH && pinUserOkStatePrev == LOW) {
511             needDisplay = true;
512             gameState = 0;
513         }
514

```

```
515     break;
516
517
518 }
519
520 pinUserOkStatePrev = pinUserOkState;
521 pinUserOtherStatePrev = pinUserOtherState;
522 }
```

Laboratoriam darbai atlikti naudotas kodas